

Objetivo

Implementar una tabla hija **convocatoria_documento** para gestionar los documentos (bases, anexos, comunicados, etc.) como **sub-recursos** de cada convocatoria. Esto permite al front decidir qué mostrar (habilitado/orden), renombrar títulos, y actualizar metadatos **sin re-subir archivos**.

¿Por qué es la mejor opción?

- **Escalabilidad:** agregar nuevos tipos de documento sin tocar la tabla principal.
- **UX del front:** consumir una **lista homogénea** (documentos) con tipo/titulo/descripcion/url/habilitado/orden.
- **Configurabilidad:** cambiar visibilidad/orden/título por endpoint sin multipart.
- **Mantenibilidad:** evitar 8+ campos duplicados en la entidad principal y lógica if/else por cada uno.

Paso 0 — Mapa de tipos actuales

Campo actual en entidad	Tipo sugerido (ENUM)
basesUrl	BASES
anexosUrl	ANEXOS
comunicado1Url	COMUNICADO1
comunicado2Url	COMUNICADO2
evaluacionCurricularUrl	EVAL_CURRICULAR
evaluacionEntrevistaUrl	EVAL_ENTREVISTA
absolucionReclamosUrl	ABSOLUCION_RECLAMOS
resultadosFinalesUrl	RESULTADOS_FINALES

Paso 1 — Crear DocumentoTipo (ENUM)

Ruta: com/mdnch/webmdnch/enums/DocumentoTipo.java

```
package com.mdnch.webmdnch.enums;  
  
public enum DocumentoTipo {  
    BASES,  
    ANEXOS,
```

```

    COMUNICADO1,
    COMUNICADO2,
    EVAL_CURRICULAR,
    EVAL_ENTREVISTA,
    ABSOLUCION_RECLAMOS,
    RESULTADOS_FINALES
}

```

Paso 2 — Crear entidad hija

ConvocatoriaDocumentoEntity

Ruta: `com/mdnch/webmdnch/entity/ConvocatoriaDocumentoEntity.java`

```

package com.mdnch.webmdnch.entity;

import com.mdnch.webmdnch.enums.DocumentoTipo;
import jakarta.persistence.*;

@Entity
@Table(name = "convocatoria_documento",
        uniqueConstraints = @UniqueConstraint(columnNames =
{"convocatoria_id", "tipo"}))
public class ConvocatoriaDocumentoEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_documento")
    private Long id;

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "convocatoria_id", nullable = false)
    private ConvocatoriaCasEntity convocatoria;

    @Enumerated(EnumType.STRING)
    @Column(name = "tipo", nullable = false, length = 40)
    private DocumentoTipo tipo;

    @Column(name = "titulo", length = 200)
    private String titulo;

    @Column(name = "descripcion", length = 500)
    private String descripcion;

    // Ruta relativa almacenada en BD, p.ej. "cas/1695412400_bases.pdf"
    @Column(name = "url", length = 500)
    private String url;
}

```

```

@Column(name = "habilitado", nullable = false)
private Boolean habilitado = Boolean.TRUE;

@Column(name = "orden")
private Integer orden;

// Getters y Setters
public Long getId() { return id; }
public void setId(Long id) { this.id = id; }

public ConvocatoriaCasEntity getConvocatoria() { return convocatoria; }
public void setConvocatoria(ConvocatoriaCasEntity convocatoria) {
this.convocatoria = convocatoria; }

public DocumentoTipo getTipo() { return tipo; }
public void setTipo(DocumentoTipo tipo) { this.tipo = tipo; }

public String getTitulo() { return titulo; }
public void setTitulo(String titulo) { this.titulo = titulo; }

public String getDescripcion() { return descripcion; }
public void setDescripcion(String descripcion) { this.descripcion =
descripcion; }

public String getUrl() { return url; }
public void setUrl(String url) { this.url = url; }

public Boolean getHabilitado() { return habilitado; }
public void setHabilitado(Boolean habilitado) { this.habilitado =
habilitado; }

public Integer getOrden() { return orden; }
public void setOrden(Integer orden) { this.orden = orden; }
}

```

Paso 3 — Relación en ConvocatoriaCasEntity

Ruta: `com/mdnch/webmdnch/entity/ConvocatoriaCasEntity.java` - Agrega la colección `documentos`. - Mantén temporalmente los campos `...Url` (para compatibilidad); los marcaremos como *deprecados* y luego los migraremos.

```

// ... imports
import java.util.ArrayList;
import java.util.List;

@Entity

```

```

@Table(name = "convocatoria_cas")
public class ConvocatoriaCasEntity {
    // ... tus campos actuales

    @OneToMany(mappedBy = "convocatoria", cascade = CascadeType.ALL,
        orphanRemoval = true)
    private List<ConvocatoriaDocumentoEntity> documentos = new ArrayList<>();

    public List<ConvocatoriaDocumentoEntity> getDocumentos() { return
documentos; }
    public void setDocumentos(List<ConvocatoriaDocumentoEntity> documentos)
{ this.documentos = documentos; }
}

```

`orphanRemoval=true` garantiza que si quitas un documento de la lista y lo guardas, se elimina la fila hija.

Paso 4 — Repositorio hijo

Ruta: `com/mdnch/webmdnch/repository/ConvocatoriaDocumentoRepository.java`

```

package com.mdnch.webmdnch.repository;

import com.mdnch.webmdnch.entity.ConvocatoriaDocumentoEntity;
import com.mdnch.webmdnch.enums.DocumentoTipo;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.List;
import java.util.Optional;

public interface ConvocatoriaDocumentoRepository extends
JpaRepository<ConvocatoriaDocumentoEntity, Long> {
    List<ConvocatoriaDocumentoEntity>
findByConvocatoriaIdOrderByOrdenAsc(Integer convocatoriaId);
    Optional<ConvocatoriaDocumentoEntity>
findByConvocatoriaIdAndTipo(Integer convocatoriaId, DocumentoTipo tipo);
}

```

Paso 5 — DTOs para front

5.1 Respuesta: lista homogénea

Ruta: `com/mdnch/webmdnch/dto/response/DocumentoItemResponse.java`

```

package com.mdnch.webmdnch.dto.response;

public class DocumentoItemResponse {
    private String tipo;           // "BASES", "ANEXOS", ...
    private String titulo;         // editable
    private String descripcion;    // opcional
    private String url;            // pública (decorada)
    private Boolean habilitado;    // front decide pintar
    private Integer orden;        // orden en UI

    public String getTipo() { return tipo; }
    public void setTipo(String tipo) { this.tipo = tipo; }
    public String getTitulo() { return titulo; }
    public void setTitulo(String titulo) { this.titulo = titulo; }
    public String getDescripcion() { return descripcion; }
    public void setDescripcion(String descripcion) { this.descripcion =
descripcion; }
    public String getUrl() { return url; }
    public void setUrl(String url) { this.url = url; }
    public Boolean getHabilitado() { return habilitado; }
    public void setHabilitado(Boolean habilitado) { this.habilitado =
habilitado; }
    public Integer getOrden() { return orden; }
    public void setOrden(Integer orden) { this.orden = orden; }
}

```

5.2 Request: configuración sin archivos

Ruta: `com/mdnch/webmdnch/dto/request/DocumentoConfigRequest.java`

```

package com.mdnch.webmdnch.dto.request;

public class DocumentoConfigRequest {
    private String tipo;           // enum como String
    private Boolean habilitado;
    private String titulo;
    private String descripcion;
    private Integer orden;

    public String getTipo() { return tipo; }
    public void setTipo(String tipo) { this.tipo = tipo; }
    public Boolean getHabilitado() { return habilitado; }
    public void setHabilitado(Boolean habilitado) { this.habilitado =
habilitado; }
    public String getTitulo() { return titulo; }
    public void setTitulo(String titulo) { this.titulo = titulo; }
    public String getDescripcion() { return descripcion; }
    public void setDescripcion(String descripcion) { this.descripcion =
}

```

```

descripcion; }
    public Integer getOrden() { return orden; }
    public void setOrden(Integer orden) { this.orden = orden; }
}

```

5.3 Extender ConvocatoriaCasRequest para incluir config

Ruta: `com/mdnch/webmdnch/dto/request/ConvocatoriaCasRequest.java`

```

// agrega:
import java.util.List;

public class ConvocatoriaCasRequest {
    // ... tus campos existentes (incluyendo MultipartFile ...)

    private List<DocumentoConfigRequest> config; // opcional

    public List<DocumentoConfigRequest> getConfig() { return config; }
    public void setConfig(List<DocumentoConfigRequest> config) { this.config
= config; }
}

```

5.4 Extender ConvocatoriaCasResponse para incluir documentos

Ruta: `com/mdnch/webmdnch/dto/response/ConvocatoriaCasResponse.java`

```

import java.util.List;

public class ConvocatoriaCasResponse {
    // ... campos actuales (puedes mantener los ...Url por transición)

    private List<DocumentoItemResponse> documentos;

    public List<DocumentoItemResponse> getDocumentos() { return documentos; }
    public void setDocumentos(List<DocumentoItemResponse> documentos) {
this.documentos = documentos; }
}

```

Paso 6 — Mapper

Puedes seguir usando tu `ConvocatoriaCasMapper` para los campos base y hacer el mapeo de documentos en el **servicio** (aplicando `toPublicUrl`). Si prefieres MapStruct, puedes crear un

`ConvocatoriaDocumentoMapper` con un `@Context` para `docUrlBase`. Para simplificar, aquí lo hacemos en el servicio.

Paso 7 — Servicio: lógica de creación/actualización/lista

Ruta: `com/mdnch/webmdnch/service/impl/ConvocatoriaCasServiceImpl.java`

7.1 — Inyectar repo hijo

```
private final ConvocatoriaDocumentoRepository docRepo;

public ConvocatoriaCasServiceImpl(ConvocatoriaCasRepository repo,
                                   ConvocatoriaCasMapper mapper,
                                   ConvocatoriaDocumentoRepository docRepo) {

    this.repo = repo;
    this.mapper = mapper;
    this.docRepo = docRepo;
}
```

7.2 — Helpers para URL pública y mapeo de documentos

```
private String toPublicUrl(String path) {
    if (path == null || path.isBlank()) return null;
    if (path.startsWith("http://") || path.startsWith("https://")) return
    path;
    String base = docUrlBase.endsWith("/") ? docUrlBase : docUrlBase + "/";
    return base + path;
}

private DocumentoItemResponse toDocResponse(ConvocatoriaDocumentoEntity e) {
    DocumentoItemResponse r = new DocumentoItemResponse();
    r.setTipo(e.getTipo().name());
    r.setTitulo(e.getTitulo());
    r.setDescripcion(e.getDescripcion());
    r.setUrl(toPublicUrl(e.getUrl()));
    r.setHabilitado(Boolean.TRUE.equals(e.getHabilitado()));
    r.setOrden(e.getOrden());
    return r;
}

private List<DocumentoItemResponse>
mapDocs(List<ConvocatoriaDocumentoEntity> docs) {
    return docs == null ? java.util.Collections.emptyList() :
        docs.stream()
            .sorted(java.util.Comparator.comparing(d -> d.getOrden()) ==
```

```

null ? 0 : d.getOrden()))
        .map(this::toDocResponse)
        .toList();
}

```

7.3 — Aplicar configuración de documentos (sin tocar archivos)

```

private void applyConfig(ConvocatoriaCasEntity entity,
List<DocumentoConfigRequest> cfgList) {
    if (cfgList == null || cfgList.isEmpty()) return;
    var byTipo = entity.getDocumentos().stream()
        .collect(java.util.stream.Collectors.toMap(d ->
d.getTipo().name(), d -> d, (a,b)->a));

    for (DocumentoConfigRequest c : cfgList) {
        if (c.getTipo() == null) continue;
        var tipo =
com.mdnch.webmdnch.enums.DocumentoTipo.valueOf(c.getTipo());
        var doc = byTipo.get(tipo.name());
        if (doc == null) {
            doc = new ConvocatoriaDocumentoEntity();
            doc.setConvocatoria(entity);
            doc.setTipo(tipo);
            entity.getDocumentos().add(doc);
        }
        if (c.getHabilitado() != null) doc.setHabilitado(c.getHabilitado());
        if (c.getTitulo() != null) doc.setTitulo(c.getTitulo());
        if (c.getDescripcion() != null)
doc.setDescripcion(c.getDescripcion());
        if (c.getOrden() != null) doc.setOrden(c.getOrden());
    }
}

```

7.4 — Guardar/Reemplazar archivos por tipo

```

private String carpetaDestino = "documentos/cas/";

private String saveFile(MultipartFile file) {
    if (file == null || file.isEmpty()) return null;
    String nombre = FileUploadUtil.guardarArchivo(file, carpetaDestino);
    return "cas/" + nombre;
}

private void replaceFile(ConvocatoriaDocumentoEntity doc, MultipartFile
nuevo) {
    if (nuevo == null || nuevo.isEmpty()) return; // no cambia
    // eliminar anterior
    if (doc.getUrl() != null) {
        FileUploadUtil.eliminarArchivo(carpetaDestino, doc.getUrl());
    }
}

```



```

    }
    doc.setUrl(saveFile(nuevo));
}

```

7.5 — En create(): crear/actualizar documentos desde archivos + aplicar config

```

@Transactional
public ConvocatoriaCasResponse create(ConvocatoriaCasRequest request) {
    if (repo.existsByCodigo(request.getCodigo())) {
        throw new
IllegalArgumentExcepcion("Ya existe una convocatoria con el código: " +
request.getCodigo());
    }
    ConvocatoriaCasEntity entity = mapper.toEntity(request);
    entity.setEstado(true);
    entity.setResponsable("Admin");
    entity.setFechaCreacion(java.time.LocalDate.now());

    // Inicializa (si deseas crear entradas aunque no haya archivos aún,
opcional)

    // Por cada campo archivo conocido, crea/actualiza doc
    java.util.Map<DocumentoTipo, MultipartFile> archivos = new
java.util.LinkedHashMap<>();
    archivos.put(DocumentoTipo.BASES, request.getBases());
    archivos.put(DocumentoTipo.ANEXOS, request.getAnexos());
    archivos.put(DocumentoTipo.COMUNICADO1, request.getComunicado1());
    archivos.put(DocumentoTipo.COMUNICADO2, request.getComunicado2());
    archivos.put(DocumentoTipo.EVAL_CURRICULAR,
request.getEvaluacionCurricular());
    archivos.put(DocumentoTipo.EVAL_ENTREVISTA,
request.getEvaluacionEntrevista());
    archivos.put(DocumentoTipo.ABSOLUCION_RECLAMOS,
request.getAbsolucionReclamos());
    archivos.put(DocumentoTipo.RESULTADOS_FINALES,
request.getResultadosFinales());

    for (var entry : archivos.entrySet()) {
        MultipartFile mf = entry.getValue();
        if (mf != null && !mf.isEmpty()) {
            ConvocatoriaDocumentoEntity doc = new
ConvocatoriaDocumentoEntity();
            doc.setConvocatoria(entity);
            doc.setTipo(entry.getKey());
            doc.setUrl(saveFile(mf));
            doc.setHabilitado(true);
            entity.getDocumentos().add(doc);
        }
    }
}

```

```

// Aplica configuración (habilitado/titulo/descripcion/orden)
applyConfig(entity, request.getConfig());

// Persistir
ConvocatoriaCasEntity saved = repo.save(entity);

// Construir response
ConvocatoriaCasResponse res = mapper.toResponse(saved);
// mantener compatibilidad si aún usas ...Url (opcional)
res.setDocumentos(mapDocs(saved.getDocumentos()));
return res;
}

```

7.6 — En update(): reemplazar archivos si vienen + aplicar config

```

@Transactional
public ConvocatoriaCasResponse update(Integer id, ConvocatoriaCasRequest
request) {
    ConvocatoriaCasEntity entity = repo.findById(id)
        .orElseThrow(() -> new
ResourceNotFoundException("Convocatoria no encontrada con ID: " + id));

    mapper.updateFromRequest(request, entity); // actualiza campos simples

    // indexar documentos existentes por tipo
    var byTipo = entity.getDocumentos().stream()
        .collect(java.util.stream.Collectors.toMap(d -> d.getTipo(), d -
> d, (a,b)->a));

    java.util.Map<DocumentoTipo, MultipartFile> archivos = new
java.util.LinkedHashMap<>();
    archivos.put(DocumentoTipo.BASES, request.getBases());
    archivos.put(DocumentoTipo.ANEXOS, request.getAnexos());
    archivos.put(DocumentoTipo.COMUNICADO1, request.getComunicado1());
    archivos.put(DocumentoTipo.COMUNICADO2, request.getComunicado2());
    archivos.put(DocumentoTipo.EVAL_CURRICULAR,
request.getEvaluacionCurricular());
    archivos.put(DocumentoTipo.EVAL_ENTREVISTA,
request.getEvaluacionEntrevista());
    archivos.put(DocumentoTipo.ABSOLUCION_RECLAMOS,
request.getAbsolucionReclamos());
    archivos.put(DocumentoTipo.RESULTADOS_FINALES,
request.getResultadosFinales());

    for (var entry : archivos.entrySet()) {
        DocumentoTipo tipo = entry.getKey();
        MultipartFile mf = entry.getValue();
        if (mf != null && !mf.isEmpty()) {
            ConvocatoriaDocumentoEntity doc = byTipo.get(tipo);
            if (doc == null) {

```

```

        doc = new ConvocatoriaDocumentoEntity();
        doc.setConvocatoria(entity);
        doc.setTipo(tipo);
        entity.getDocumentos().add(doc);
        byTipo.put(tipo, doc);
    }
    replaceFile(doc, mf); // elimina anterior y guarda nuevo
    // no cambiamos habilitado/título/orden aquí (lo hará config si
llega)
    }
}

// Aplica configuración
applyConfig(entity, request.getConfig());

entity.setResponsable("Admin Update");
entity.setFechaModificacion(java.time.LocalDate.now());

ConvocatoriaCasEntity updated = repo.save(entity);

ConvocatoriaCasResponse res = mapper.toResponse(updated);
res.setDocumentos(mapDocs(updated.getDocumentos()));
return res;
}

```

7.7 — En getAll()/findById(): llenar documentos

```

public List<ConvocatoriaCasResponse> getAll() {
    return repo.findAll().stream()
        .map(mapper::toResponse)
        .peek(r -> {
            // cargar documentos vía repo hijo (opcional) o por fetch si
LAZY/EAGER
        })
        .map(r -> {
            var e = repo.findById(r.getId()).orElseThrow();
            r.setDocumentos(mapDocs(e.getDocumentos()));
            // si mantienes ...Url, puedes seguir decorándolas, pero
ideal migrar al arreglo
            return r;
        })
        .toList();
}

public ConvocatoriaCasResponse findById(Integer id) {
    ConvocatoriaCasEntity entity = repo.findById(id)
        .orElseThrow(() -> new
ResourceNotFoundException("Convocatoria no encontrada con ID: " + id));
    ConvocatoriaCasResponse r = mapper.toResponse(entity);
    r.setDocumentos(mapDocs(entity.getDocumentos()));
}

```

```

        return r;
    }

```

Nota: Para eficiencia, puedes usar `@EntityGraph` o `JOIN FETCH` si `documentos` es LAZY.

7.8 — En delete(): borrar usando los hijos

```

@Transactional
public void delete(Integer id) {
    ConvocatoriaCasEntity entity = repo.findById(id)
        .orElseThrow(() -> new
ResourceNotFoundException("Convocatoria no encontrada con ID: " + id));

    for (var d : entity.getDocumentos()) {
        FileUploadUtil.eliminarArchivo(carpetasDestino, d.getUrl());
    }
    // Por orphanRemoval=true, al hacer repo.deleteByid se eliminan hijos
    repo.deleteById(id);
}

```

Paso 8 — Endpoint dedicado para configurar visibilidad/orden

Ruta: en `ConvocatoriaCasController`

```

@PatchMapping(value =("/{id}/documentos/config"))
public ResponseEntity<ResponseBase<ConvocatoriaCasResponse>>
actualizarConfig(
    @PathVariable Integer id,
    @RequestBody List<DocumentoConfigRequest> config) {

    ConvocatoriaCasEntity entity = serviceInternalFind(id); // o usa el
service existente
    // si no exposes service interno, crea método en service:
updateDocsConfig(id, config)

    // ejemplo directo (mejor mover a service):
    entity = repo.findById(id).orElseThrow(() -> new
ResourceNotFoundException("Convocatoria no encontrada"));
    applyConfig(entity, config); // la misma del servicio
    entity.setFechaModificacion(java.time.LocalDate.now());
    var saved = repo.save(entity);

    ConvocatoriaCasResponse r = mapper.toResponse(saved);
}

```

```

        r.setDocumentos(mapDocs(saved.getDocumentos()));

        return ResponseEntity.ok(new ResponseBase<>(true, "Configuración
actualizada", r));
    }

```

Recomendado: Implementar `updateDocsConfig(id, config)` en el `Service` y llamarlo desde el controller para mantener el patrón capa de servicio.

Paso 9 — Migración SQL (opcional pero recomendado)

1) Crear tabla hija:

```

CREATE TABLE convocatoria_documento (
    id_documento SERIAL PRIMARY KEY,
    convocatoria_id INTEGER NOT NULL,
    tipo VARCHAR(40) NOT NULL,
    titulo VARCHAR(200),
    descripcion VARCHAR(500),
    url VARCHAR(500),
    habilitado BOOLEAN NOT NULL DEFAULT TRUE,
    orden INTEGER,
    CONSTRAINT fk_convocatoria_doc FOREIGN KEY (convocatoria_id)
        REFERENCES convocatoria_cas(id_convocatoria) ON DELETE CASCADE,
    CONSTRAINT uq_convocatoria_tipo UNIQUE (convocatoria_id, tipo)
);

```

2) Migrar datos desde columnas antiguas (ejemplo en SQL genérico):

```

-- BASES
INSERT INTO convocatoria_documento (convocatoria_id, tipo, url, habilitado,
orden)
SELECT id_convocatoria, 'BASES', bases_url, TRUE, 1
FROM convocatoria_cas
WHERE bases_url IS NOT NULL;

-- ANEXOS
INSERT INTO convocatoria_documento (convocatoria_id, tipo, url, habilitado,
orden)
SELECT id_convocatoria, 'ANEXOS', anexos_url, TRUE, 2
FROM convocatoria_cas
WHERE anexos_url IS NOT NULL;

```

```
-- ... y así para los demás tipos ...
```

3) (**Posterior**): eliminar columnas `..._url` de la tabla principal cuando el front ya consuma documentos.

Paso 10 — Buenas prácticas y consideraciones

- **Transacciones:** BD y disco no son atómicos. Maneja errores de archivo con cuidado. Opcional: usar `TransactionSynchronization` para borrar tras commit.
- **Seguridad de archivos:** valida MIME/extensión/tamaño y sanitiza `originalFilename` (evitar path traversal).
- `docUrlBase`: mantener `http(s)://host/documentos/` (con `/` final).
- **Orden:** define defaults (1..n) por tipo para UI estable.
- `LocalDate` vs `LocalDateTime`: usa `LocalDateTime` si necesitas auditar hora exacta (zona `America/Lima`).
- **Back-compat:** durante transición, sigue decorando los `...Url` si el front aún los usa, pero impulsa el consumo de `documentos`.

Ejemplo de respuesta final para el front

```
{
  "success": true,
  "message": "Convocatoria obtenida con éxito",
  "data": {
    "id": 12,
    "codigo": "CAS-015-2025",
    "convocatoria": "Analista SIGA",
    "area": "Logística",
    "vacantes": 2,
    "postulacion": "https://forms.gle/xyz",
    "estado": true,
    "responsable": "Juan Pérez",
    "fechaCreacion": "2025-09-20",
    "fechaModificacion": "2025-09-23",
    "documentos": [
      { "tipo": "BASES", "titulo": "Bases del proceso", "descripcion": null,
        "url": "https://host/documentos/cas/1695412345_bases.pdf", "habilitado":
        true, "orden": 1 },
      { "tipo": "ANEXOS", "titulo": "Anexos", "descripcion": null, "url":
        "https://host/documentos/cas/1695412400_anexos.pdf", "habilitado": false,
        "orden": 2 },
      { "tipo": "RESULTADOS_FINALES", "titulo": "Resultados finales",
        "descripcion": "Publicación oficial", "url": null, "habilitado": false,
        "orden": 99 }
    ]
  }
}
```

```

    }
}
}

```

Con esto, el front puede hacer: `docs.filter(d => d.habilitado).sort(...)` y renderizar sin lógica extra.

Siguientes pasos rápidos

1. Crear `DocumentoTipo`, `ConvocatoriaDocumentoEntity`, repo y DTOs.
2. Agregar relación `documentos` en la entidad padre.
3. Ajustar `service` (create/update/find/delete) con helpers provistos.
4. Añadir endpoint `PATCH /{id}/documentos/config`.
5. Ejecutar la migración SQL y, tras la transición, eliminar columnas antiguas.

Fase 2 — Adaptar el *Service* para usar la tabla hija y devolver documentos

En esta fase no tocamos aún el **Controller** (salvo para inyectar el repo nuevo en el service a través del constructor) ni removemos tus campos `...Url`. Solo: 1) Inyectamos `ConvocatoriaDocumentoRepository`. 2) Creamos *helpers* para mapear `ConvocatoriaDocumentoEntity` → `DocumentoItemResponse` con `toPublicUrl`. 3) En `create(...)` guardamos documentos hijo si vienen archivos y aplicamos `config` (si viene). 4) En `update(...)` reemplazamos archivos por tipo y aplicamos `config`. 5) En `findById(...)` y `getAll(...)` llenamos `response.setDocumentos(...)`. 6) En `delete(...)` eliminamos los archivos de los hijos (además de lo que ya tenías).

Nota: mantenemos compatibilidad con tus campos `...Url` actuales para que el front no se rompa durante la transición.

2.1 — Inyección del repo hijo en el service

Archivo: `ConvocatoriaCasServiceImpl.java`

```

// imports nuevos
import com.mdnch.webmdnch.repository.ConvocatoriaDocumentoRepository;
import com.mdnch.webmdnch.entity.ConvocatoriaDocumentoEntity;
import com.mdnch.webmdnch.enums.DocumentoTipo;
import com.mdnch.webmdnch.dto.request.DocumentoConfigRequest;
import com.mdnch.webmdnch.dto.response.DocumentoItemResponse;

// ...

private final ConvocatoriaDocumentoRepository docRepo;

```

```

public ConvocatoriaCasServiceImpl(ConvocatoriaCasRepository repo,
                                   ConvocatoriaCasMapper mapper,
                                   ConvocatoriaDocumentoRepository docRepo) {

    this.repo = repo;
    this.mapper = mapper;
    this.docRepo = docRepo;
}

```

Si usas Lombok `@RequiredArgsConstructor`, basta con declarar `final` y remover el constructor manual.

2.2 — Helpers para URL pública y mapeo de documentos

Pégalos al final de la clase (puedes reutilizar tu `toPublicUrl` existente):

```

private DocumentoItemResponse toDocResponse(ConvocatoriaDocumentoEntity e) {
    DocumentoItemResponse r = new DocumentoItemResponse();
    r.setTipo(e.getTipo().name());
    r.setTitulo(e.getTitulo());
    r.setDescripcion(e.getDescripcion());
    r.setUrl(toPublicUrl(e.getUrl()));
    r.setHabilitado(Boolean.TRUE.equals(e.getHabilitado()));
    r.setOrden(e.getOrden());
    return r;
}

private List<DocumentoItemResponse>
mapDocs(Collection<ConvocatoriaDocumentoEntity> docs) {
    return docs == null ? java.util.Collections.emptyList() :
        docs.stream()
            .sorted(java.util.Comparator.comparing(d -> d.getOrden() ==
null ? 0 : d.getOrden()))
            .map(this::toDocResponse)
            .collect(java.util.stream.Collectors.toList());
}

private void applyConfig(ConvocatoriaCasEntity entity,
List<DocumentoConfigRequest> cfgList) {
    if (cfgList == null || cfgList.isEmpty()) return;

    Map<DocumentoTipo, ConvocatoriaDocumentoEntity> byTipo =
entity.getDocumentos().stream()
    .collect(java.util.stream.Collectors.toMap(ConvocatoriaDocumentoEntity::getTipo,
d -> d, (a,b)->a));

    for (DocumentoConfigRequest c : cfgList) {
        if (c.getTipo() == null) continue;
        DocumentoTipo tipo = DocumentoTipo.valueOf(c.getTipo());

```



```

        ConvocatoriaDocumentoEntity doc = byTipo.get(tipo);
        if (doc == null) {
            doc = new ConvocatoriaDocumentoEntity();
            doc.setConvocatoria(entity);
            doc.setTipo(tipo);
            entity.getDocumentos().add(doc);
            byTipo.put(tipo, doc);
        }
        if (c.getHabilitado() != null) doc.setHabilitado(c.getHabilitado());
        if (c.getTitulo() != null) doc.setTitulo(c.getTitulo());
        if (c.getDescripcion() != null)
doc.setDescripcion(c.getDescripcion());
        if (c.getOrden() != null) doc.setOrden(c.getOrden());
    }
}

private String saveFile(MultipartFile file) {
    if (file == null || file.isEmpty()) return null;
    String nombre = FileUploadUtil.guardarArchivo(file, carpetaDestino);
    return "cas/" + nombre; // ruta relativa que luego se decora
}

private void replaceFile(ConvocatoriaDocumentoEntity doc, MultipartFile
nuevo) {
    if (nuevo == null || nuevo.isEmpty()) return;
    if (doc.getUrl() != null) {
        FileUploadUtil.eliminarArchivo(carpetaDestino, doc.getUrl());
    }
    doc.setUrl(saveFile(nuevo));
}
}

```

2.3 — `create(...)`: guardar hijos por archivos + aplicar config

Reemplaza tu método `create` por esta versión (conserva el resto de tu lógica original):

```

@Override
@Transactional
public ConvocatoriaCasResponse create(ConvocatoriaCasRequest request) {
    if (repo.existsByCodigo(request.getCodigo())) {
        throw new
IllegalArgumentException("Ya existe una convocatoria con el código: " +
request.getCodigo());
    }

    ConvocatoriaCasEntity entity = mapper.toEntity(request);
    entity.setEstado(true);
    entity.setResponsable("Admin");
    entity.setFechaCreacion(LocalDate.now());
}

```

```

        // Construir mapa de archivos por tipo
        Map<DocumentoTipo, MultipartFile> archivos = new
java.util.LinkedHashMap<>();
        archivos.put(DocumentoTipo.BASES, request.getBases());
        archivos.put(DocumentoTipo.ANEXOS, request.getAnexos());
        archivos.put(DocumentoTipo.COMUNICADO1, request.getComunicado1());
        archivos.put(DocumentoTipo.COMUNICADO2, request.getComunicado2());
        archivos.put(DocumentoTipo.EVAL_CURRICULAR,
request.getEvaluacionCurricular());
        archivos.put(DocumentoTipo.EVAL_ENTREVISTA,
request.getEvaluacionEntrevista());
        archivos.put(DocumentoTipo.ABSOLUCION_RECLAMOS,
request.getAbsolucionReclamos());
        archivos.put(DocumentoTipo.RESULTADOS_FINALES,
request.getResultadosFinales());

        for (Map.Entry<DocumentoTipo, MultipartFile> entry :
archivos.entrySet()) {
            MultipartFile mf = entry.getValue();
            if (mf != null && !mf.isEmpty()) {
                ConvocatoriaDocumentoEntity doc = new
ConvocatoriaDocumentoEntity();
                doc.setConvocatoria(entity);
                doc.setTipo(entry.getKey());
                doc.setUrl(saveFile(mf));
                doc.setHabilitado(true);
                entity.getDocumentos().add(doc);
            }
        }

        // Aplica config (habilitado, titulo, descripcion, orden) si llega en
request
        applyConfig(entity, request.getConfig());

        ConvocatoriaCasEntity saved = repo.save(entity);

        ConvocatoriaCasResponse res = mapper.toResponse(saved);

        // Mantén (si quieres) la decoración de tus ...Url existentes aquí

        // Nuevo: setear lista homogénea de documentos
        res.setDocumentos(mapDocs(saved.getDocumentos()));
        return res;
    }

```

2.4 — `update(...)`: reemplazar archivo si llega + aplicar config

```

@Override
@Transactional
public ConvocatoriaCasResponse update(Integer id, ConvocatoriaCasRequest

```

```

request) {
    ConvocatoriaCasEntity entity = repo.findById(id)
        .orElseThrow(() -> new
ResourceNotFoundException("Convocatoria no encontrada con ID: " + id));

    mapper.updateFromRequest(request,
entity); // actualiza campos simples no nulos

    // Indexa documentos existentes por tipo
    Map<DocumentoTipo, ConvocatoriaDocumentoEntity> byTipo =
entity.getDocumentos().stream()
        .collect(java.util.stream.Collectors.toMap(ConvocatoriaDocumentoEntity::getTipo,
d -> d, (a,b)->a));

    Map<DocumentoTipo, MultipartFile> archivos = new
java.util.LinkedHashMap<>();
    archivos.put(DocumentoTipo.BASES, request.getBases());
    archivos.put(DocumentoTipo.ANEXOS, request.getAnexos());
    archivos.put(DocumentoTipo.COMUNICADO1, request.getComunicado1());
    archivos.put(DocumentoTipo.COMUNICADO2, request.getComunicado2());
    archivos.put(DocumentoTipo.EVAL_CURRICULAR,
request.getEvaluacionCurricular());
    archivos.put(DocumentoTipo.EVAL_ENTREVISTA,
request.getEvaluacionEntrevista());
    archivos.put(DocumentoTipo.ABSOLUCION_RECLAMOS,
request.getAbsolucionReclamos());
    archivos.put(DocumentoTipo.RESULTADOS_FINALES,
request.getResultadosFinales());

    for (Map.Entry<DocumentoTipo, MultipartFile> entry :
archivos.entrySet()) {
        DocumentoTipo tipo = entry.getKey();
        MultipartFile mf = entry.getValue();
        if (mf != null && !mf.isEmpty()) {
            ConvocatoriaDocumentoEntity doc = byTipo.get(tipo);
            if (doc == null) {
                doc = new ConvocatoriaDocumentoEntity();
                doc.setConvocatoria(entity);
                doc.setTipo(tipo);
                entity.getDocumentos().add(doc);
                byTipo.put(tipo, doc);
            }
            replaceFile(doc, mf); // elimina anterior y guarda nuevo
        }
    }

    // Aplica configuración (habilitado/titulo/descripcion/orden)
    applyConfig(entity, request.getConfig());

    entity.setResponsable("Admin Update");
    entity.setFechaModificacion(LocalDate.now());
}

```

```

ConvocatoriaCasEntity updated = repo.save(entity);

ConvocatoriaCasResponse res = mapper.toResponse(updated);
res.setDocumentos(mapDocs(updated.getDocumentos()));
return res;
}

```

2.5 — findById(...) y getAll(...): añadir documentos al response

```

@Override
public ConvocatoriaCasResponse findById(Integer id) {
    ConvocatoriaCasEntity entity = repo.findById(id)
        .orElseThrow(() -> new
ResourceNotFoundException("Convocatoria no encontrada con ID: " + id));
    ConvocatoriaCasResponse r = mapper.toResponse(entity);
    r.setDocumentos(mapDocs(entity.getDocumentos()));
    return r;
}

@Override
public List<ConvocatoriaCasResponse> getAll() {
    return repo.findAll().stream()
        .map(e -> {
            ConvocatoriaCasResponse r = mapper.toResponse(e);
            r.setDocumentos(mapDocs(e.getDocumentos()));
            return r;
        })
        .collect(Collectors.toList());
}

```

Si ves consultas N+1 por LAZY en `getAll()`, luego optimizamos con `@EntityGraph` o `JOIN FETCH`.

2.6 — delete(...): borrar archivos de los hijos

```

@Override
@Transactional
public void delete(Integer id) {
    ConvocatoriaCasEntity entity = repo.findById(id)
        .orElseThrow(() -> new
ResourceNotFoundException("Convocatoria no encontrada con ID: " + id));

    // Borrar archivos de documentos hijos
    for (ConvocatoriaDocumentoEntity d : entity.getDocumentos()) {
        FileUploadUtil.eliminarArchivo(carpetaDestino, d.getUrl());
    }
}

```

```

    }

    // (opcional) mantener lo que ya eliminabas para campos ...Url durante
    transición
    FileUploadUtil.eliminarArchivo(carpetadestino, entity.getBasesUrl());
    FileUploadUtil.eliminarArchivo(carpetadestino, entity.getAnexosUrl());
    FileUploadUtil.eliminarArchivo(carpetadestino,
entity.getComunicado1Url());
    FileUploadUtil.eliminarArchivo(carpetadestino,
entity.getComunicado2Url());
    FileUploadUtil.eliminarArchivo(carpetadestino,
entity.getEvaluacionCurricularUrl());
    FileUploadUtil.eliminarArchivo(carpetadestino,
entity.getEvaluacionEntrevistaUrl());
    FileUploadUtil.eliminarArchivo(carpetadestino,
entity.getAbsolucionReclamosUrl());
    FileUploadUtil.eliminarArchivo(carpetadestino,
entity.getResultadosFinalesUrl());

    repo.deleteById(id); // ON DELETE CASCADE también elimina en BD
}

```

¿Qué ganamos con esto ya mismo?

- Tu API actual **sigue funcionando** (mantienes los campos `...Url`).
- Empiezas a entregar al front un **array homogéneo** `documentos` con `tipo/titulo/descripcion/url/habilitado/orden`.
- Ya puedes **configurar visibilidad/orden/títulos** pasando `config` dentro del mismo `POST/PATCH multipart` **sin re-subir** archivos.

Siguiente fase (Fase 3): añadir un endpoint dedicado `PATCH /api/convocatorias/{id}/documentos/config` para que el front pueda togglear visibilidad y ordenar sin multipart y sin tocar otros campos. Además, optimizamos consultas y preparamos la migración para eliminar los `...Url` antiguos cuando el front ya use `documentos`.

Fase 3 — Endpoint dedicado para configurar documentos (sin multipart)

Objetivo: permitir al front **activar/desactivar**, **renombrar** y **ordenar** documentos **sin** re-subir archivos ni tocar otros campos de la convocatoria.

3.1 — Contrato del endpoint

- **Ruta:** `PATCH /api/convocatorias/{id}/documentos/config`
- **Body (JSON):** `List<DocumentoConfigRequest>`

- **Efecto:** crea filas hijo si no existen (sin URL) y actualiza `habilitado/titulo/descripcion/orden`.
- **Idempotente:** múltiples llamadas con los mismos datos no cambian el resultado.

Request DTO (ya creado)

```
DocumentoConfigRequest { tipo, habilitado, titulo, descripcion, orden }
```

Respuesta

`ResponseBase<ConvocatoriaCasResponse>` incluyendo la lista `documentos` actualizada (URLs decoradas) para que el front refresque su UI.

3.2 — Service API

Archivo: `ConvocatoriaCasService.java`

```
ConvocatoriaCasResponse updateDocsConfig(Integer id,
List<DocumentoConfigRequest> config);
```

Archivo: `ConvocatoriaCasServiceImpl.java`

```
@Override
@Transactional
public ConvocatoriaCasResponse updateDocsConfig(Integer id,
List<DocumentoConfigRequest> config) {
    ConvocatoriaCasEntity entity = repo.findById(id)
        .orElseThrow(() -> new
ResourceNotFoundException("Convocatoria no encontrada con ID: " + id));

    applyConfig(entity, config); // usa el helper de Fase 2
    entity.setFechaModificacion(LocalDate.now());

    ConvocatoriaCasEntity saved = repo.save(entity);

    ConvocatoriaCasResponse res = mapper.toResponse(saved);
    res.setDocumentos(mapDocs(saved.getDocumentos()));
    return res;
}
```

Por qué `@Transactional`: si varios documentos se actualizan en el mismo parche, se garantiza atomicidad.

3.3 — Controller

Archivo: ConvocatoriaCasController.java

```
@PatchMapping(value =("/{id}/documentos/config", consumes = "application/
json")
public ResponseEntity<ResponseBase<ConvocatoriaCasResponse>>
actualizarConfig(
    @PathVariable Integer id,
    @RequestBody List<DocumentoConfigRequest> config) {
    ConvocatoriaCasResponse data = service.updateDocsConfig(id, config);
    return ResponseEntity.ok(new ResponseBase<>(true, "Configuración de
documentos actualizada", data));
}
```

3.4 — Validaciones recomendadas

1) **Enum válido:** en `applyConfig` ya usamos `DocumentoTipo.valueOf(c.getTipo())`. Considera capturar `IllegalArgumentException` para devolver 400 con mensaje claro si llega un `tipo` desconocido. 2) **Orden:** si `orden` es negativo o nulo, normaliza a `0` o `null`. 3) **Límites:** `titulo` (`<=200`) y `descripcion` (`<=500`); Spring valida por tamaño en BD, pero puedes agregar Bean Validation si quieres:

```
// en DocumentoConfigRequest (opcional si usas @Valid)
@Size(max = 200) private String titulo;
@Size(max = 500) private String descripcion;
```

4) **Seguridad:** si tu proyecto tiene autenticación, valida permisos (p.ej., rol "ADMIN") a nivel de endpoint usando `@PreAuthorize("hasRole('ADMIN')")` o similar.

3.5 — Ejemplos de uso

3.5.1 — Activar/ocultar y ordenar

PATCH `/api/convocatorias/12/documentos/config`

```
[
  { "tipo": "BASES", "habilitado": true, "orden": 1, "titulo": "Bases del
proceso" },
  { "tipo": "ANEXOS", "habilitado": false, "orden": 2 },
  { "tipo": "RESULTADOS_FINALES", "habilitado": false, "orden": 99,
"descripcion": "Se publicará tras entrevista" }
]
```

Respuesta (resumen): `data.documentos` refleja los cambios; si no existía la fila, se crea sin `url`.

3.5.2 — Solo cambiar títulos (sin tocar habilitado ni orden)

```
[
  { "tipo": "COMUNICADO1", "titulo": "Comunicado oficial N° 1" },
  { "tipo": "COMUNICADO2", "titulo": "Comunicado oficial N° 2" }
]
```

3.6 — Buenas prácticas

- **No borrar URL al deshabilitar:** `habilitado=false` solo oculta en front; conserva el archivo para histórico.
- **Crear si no existe:** config puede crear el doc "placeholder" (sin `url`) para alistar el slot en UI.
- **Idempotencia:** invocar varias veces con los mismos datos no debe producir cambios adicionales.
- **Auditoría:** actualiza `fechaModificacion` y `responsable` si ya tienes el usuario autenticado.

3.7 — Optimización de consultas (opcional)

Para evitar N+1 al listar:

```
// En el repo padre, define un EntityGraph
@EntityGraph(attributePaths = "documentos")
List<ConvocatoriaCasEntity> findAll();

@EntityGraph(attributePaths = "documentos")
Optional<ConvocatoriaCasEntity> findById(Integer id);
```

O crea métodos alternos `findAllWithDocs()` / `findByIdWithDocs(...)` usando `@Query("SELECT c FROM ConvocatoriaCasEntity c LEFT JOIN FETCH c.documentos WHERE c.id = :id")`.

3.8 — Errores comunes y cómo manejarlos

- **Tipo inválido** → devolver 400: `"tipo 'X' no es válido. Valores permitidos: BASES, ANEXOS, ..."`.
- **Convocatoria inexistente** → 404 ya manejado con `ResourceNotFoundException`.
- **Violación UNIQUE (convocatoria_id, tipo)** → si dos threads crean el mismo tipo: manejar `DataIntegrityViolationException` → 409 Conflicto con mensaje claro.

Con esto, tu front ya puede **togglear visibilidad** y **ordenar** sin multipart, y tu back mantiene la fuente de la verdad en la tabla hija.

Fase 4 — Performance (EntityGraph / JOIN FETCH) y consistencia

Buscamos evitar el problema **N+1** al obtener `documentos` junto con la convocatoria, y garantizar consistencia cuando listamos muchas convocatorias.

4.1 — Opción A: `@EntityGraph`

Archivo: `ConvocatoriaCasRepository.java`

```
import org.springframework.data.jpa.repository.EntityGraph;

public interface ConvocatoriaCasRepository extends
    JpaRepository<ConvocatoriaCasEntity, Integer> {

    boolean existsByCodigo(String codigo);

    @Override
    @EntityGraph(attributePaths = {"documentos"})
    List<ConvocatoriaCasEntity> findAll();

    @Override
    @EntityGraph(attributePaths = {"documentos"})
    Optional<ConvocatoriaCasEntity> findById(Integer id);
}
```

Ventajas: simple, declarativo. **Nota:** Por defecto hace `LEFT JOIN` sobre `documentos`. Si necesitas ordenar por `orden`, ordénalo en la capa service tras mapear.

4.2 — Opción B: `JOIN FETCH` específico

Archivo: `ConvocatoriaCasRepository.java`

```
@Query("SELECT c FROM ConvocatoriaCasEntity c LEFT JOIN FETCH c.documentos
WHERE c.id = :id")
Optional<ConvocatoriaCasEntity> findByIdWithDocs(@Param("id") Integer id);

@Query("SELECT DISTINCT c FROM ConvocatoriaCasEntity c LEFT JOIN FETCH
c.documentos")
List<ConvocatoriaCasEntity> findAllWithDocs();
```

Ventajas: control fino; `DISTINCT` evita duplicados por el JOIN. **Uso:** llama a estos métodos desde el service en lugar de los `findById/findAll` estándar.

4.3 — Ajustes en Service para usar la estrategia elegida

Si usas **EntityGraph** (Opción A), no cambias nada en el service (ya resolvimos N+1). Si usas **JOIN FETCH** (Opción B):

```
public List<ConvocatoriaCasResponse> getAll() {
    return repo.findAllWithDocs().stream()
        .map(e -> {
            ConvocatoriaCasResponse r = mapper.toResponse(e);
            r.setDocumentos(mapDocs(e.getDocumentos()));
            return r;
        })
        .collect(Collectors.toList());
}

public ConvocatoriaCasResponse findById(Integer id){
    var entity = repo.findByIdWithDocs(id)
        .orElseThrow(() -> new ResourceNotFoundException("Convocatoria no encontrada con ID: " + id));
    ConvocatoriaCasResponse r = mapper.toResponse(entity);
    r.setDocumentos(mapDocs(entity.getDocumentos()));
    return r;
}
```

4.4 — Orden estable en memoria

Aunque uses `ORDER BY` en BD, es razonable aplicar orden en memoria (por si falta). Nuestro helper `mapDocs` ya ordena por `orden` (null→0). Si prefieres garantizar en BD:

```
@OneToMany(mappedBy = "convocatoria", cascade = CascadeType.ALL,
    orphanRemoval = true)
@OrderBy("orden ASC NULLS LAST") // para Postgres; en MySQL: "orden ASC"
private List<ConvocatoriaDocumentoEntity> documentos = new ArrayList<>();
```

Fase 5 — Migración desde campos `...Url` a tabla hija

Objetivo: mover los datos existentes a `convocatoria_documento`, mantener **compatibilidad temporal**, y finalmente **remove** las columnas antiguas.

5.1 — Script de migración de datos

Crea un archivo Flyway, p.ej. `V1_3__migrate_convocatoria_urls_to_child.sql`.

MySQL

```
-- BASES
INSERT INTO convocatoria_documento (convocatoria_id, tipo, url, habilitado,
orden)
SELECT id_convocatoria, 'BASES', bases_url, 1, 1
FROM convocatoria_cas WHERE bases_url IS NOT NULL;

-- ANEXOS
INSERT INTO convocatoria_documento (convocatoria_id, tipo, url, habilitado,
orden)
SELECT id_convocatoria, 'ANEXOS', anexos_url, 1, 2
FROM convocatoria_cas WHERE anexos_url IS NOT NULL;

-- COMUNICADO1
INSERT INTO convocatoria_documento (convocatoria_id, tipo, url, habilitado,
orden)
SELECT id_convocatoria, 'COMUNICADO1', comunicado1_url, 1, 10
FROM convocatoria_cas WHERE comunicado1_url IS NOT NULL;

-- COMUNICADO2
INSERT INTO convocatoria_documento (convocatoria_id, tipo, url, habilitado,
orden)
SELECT id_convocatoria, 'COMUNICADO2', comunicado2_url, 1, 11
FROM convocatoria_cas WHERE comunicado2_url IS NOT NULL;

-- EVAL_CURRICULAR
INSERT INTO convocatoria_documento (convocatoria_id, tipo, url, habilitado,
orden)
SELECT id_convocatoria, 'EVAL_CURRICULAR', eval_curricular_url, 1, 20
FROM convocatoria_cas WHERE eval_curricular_url IS NOT NULL;

-- EVAL_ENTREVISTA
INSERT INTO convocatoria_documento (convocatoria_id, tipo, url, habilitado,
orden)
SELECT id_convocatoria, 'EVAL_ENTREVISTA', eval_entrevista_url, 1, 21
FROM convocatoria_cas WHERE eval_entrevista_url IS NOT NULL;

-- ABSOLUCION_RECLAMOS
INSERT INTO convocatoria_documento (convocatoria_id, tipo, url, habilitado,
orden)
SELECT id_convocatoria, 'ABSOLUCION_RECLAMOS', absolucion_reclamos_url, 1, 30
FROM convocatoria_cas WHERE absolucion_reclamos_url IS NOT NULL;

-- RESULTADOS_FINALES
INSERT INTO convocatoria_documento (convocatoria_id, tipo, url, habilitado,
```

```
orden)
SELECT id_convocatoria, 'RESULTADOS_FINALES', resultados_finales_url, 1, 40
FROM convocatoria_cas WHERE resultados_finales_url IS NOT NULL;
```

PostgreSQL (idéntico salvo booleanos `TRUE` y `NULLS LAST` si ordenas):

```
-- Cambia 1 por TRUE si prefieres
```

Orden sugerido: deja huecos (1,2,10,11,20,21,30,40) para insertar nuevos tipos en el futuro sin reordenar todo.

5.2 — Compatibilidad temporal en el código

Mientras el front migra a `data.documentos`, puedes mantener los getters antiguos rellenando desde la lista hija **solo en respuesta** (no guardes de vuelta):

En `withPublicUrls(...)` (si aún lo usas): - Puedes dejarlo hasta que el front migre. - En paralelo, asegúrate de que `res.setDocumentos(mapDocs(...))` ya esté funcionando.

5.3 — Remoción de columnas antiguas

Una vez que el front consuma únicamente `documentos`: 1) Elimina del **código** los campos `...Url` de `ConvocatoriaCasEntity`, `ConvocatoriaCasResponse`, mapper y service. 2) Crea una migración Flyway `V1_4__drop_old_convocatoria_urls.sql`:

MySQL

```
ALTER TABLE convocatoria_cas
DROP COLUMN bases_url,
DROP COLUMN anexos_url,
DROP COLUMN comunicado1_url,
DROP COLUMN comunicado2_url,
DROP COLUMN eval_curricular_url,
DROP COLUMN eval_entrevista_url,
DROP COLUMN absolucion_reclamos_url,
DROP COLUMN resultados_finales_url;
```

PostgreSQL

```
ALTER TABLE convocatoria_cas
DROP COLUMN bases_url,
DROP COLUMN anexos_url,
DROP COLUMN comunicado1_url,
DROP COLUMN comunicado2_url,
DROP COLUMN eval_curricular_url,
```

```
DROP COLUMN eval_entrevista_url,  
DROP COLUMN absolucion_reclamos_url,  
DROP COLUMN resultados_finales_url;
```

Tip: Haz backup antes de la eliminación definitiva y valida que todas las vistas/UI consumen el arreglo `documentos`.

5.4 — Pruebas recomendadas

- **Create con algunos archivos:** verificar que se creen filas en la hija con `url` y `habilitado=true` por defecto.
- **Update sin archivos + config:** cambiar solo `habilitado/orden/titulo` y confirmar que las `url` no cambian.
- **Update con archivo en un tipo:** reemplazar físico y mantener metadatos (salvo que config los cambie).
- **PATCH /documentos/config:** crear placeholder sin url para un tipo nuevo; front debe mostrarlo solo si `habilitado=true`.
- **Delete:** verificar que se borren del disco los hijos; revisar ON DELETE CASCADE.
- **Listar muchas:** con EntityGraph/JOIN FETCH confirmar que no haya N+1 (monitor de queries o logs).

5.5 — Limpieza final y checklist

- [] Front migrado a `data.documentos`.
- [] Campos `...Url` removidos del código.
- [] Migración `DROP COLUMN` aplicada.
- [] Logs de file I/O revisados.
- [] Documentado en README cómo agregar nuevos tipos (`DocumentoTipo`) y cómo configurarlos.

Con esto cierras el rediseño: datos normalizados, API más limpia para el front y mejor performance. ¿Avanzamos con una **Fase 6** corta para seguridad de archivos (validación de MIME/extensión/tamaño y saneamiento de nombres) y auditoría (responsable desde sesión de Spring Security)?